Exercício - Lista 4 - capítulos 11 a 13	
Curso: Engenharia da Computação	**Data:** 11/07/2025
Disciplina: Sistemas Operacionais	**Nota:**
Professor: Michelle Hanne Soares de Andrade	
Nome:	

1. Considere as afirmações a seguir sobre **comunicação interprocessos**, **região crítica, condição de corrida, exclusão mútua, semáforos e monitores.** Informe para cada uma, se são verdadeiras ou falsas.

a) ( ) Semáforos são usados para evitar que dois ou mais processos acessem sua seção crítica simultaneamente. Assim, podem garantir a exclusão mútua, onde cada processo deverá sinalizar imediatamente antes e, imediatamente após, usar um recurso comum a ambos.

b) ( ) A condição de exclusão mútua, em que um recurso ou está associado a um único processo ou está disponível, é suficiente para a ocorrência de impasses ou deadlocks.

c) ( ) Para que seja possível a ocorrência de impasses ou deadlocks, é necessário que o sistema tenha a possibilidade de preempção.

d) ( ) Um princípio obrigatório da boa exclusão mútua é que somente um processo pode estar na região crítica a cada momento.

e) ( ) Um princípio desejável mas não obrigatório da boa exclusão mútua é garantir que nenhum processo fora da região crítica impeça outros de entrarem em suas regiões críticas.

f) ( ) e) Uma forma de implementar a espera ocupada é desabilitar interrupções, embora essa situação possa gerar alguns problemas.

2. Sobre as afirmações a seguir, **relativas aos mecanismos de coordenação**, indique quais são incorretas, justificando sua resposta:

a)( ) A estratégia de inibir interrupções para evitar condições de disputa funciona em sistemas multi-processados.

b)( ) Os mecanismos de controle de entrada nas regiões críticas provêem exclusão mútua no acesso às mesmas.

c)( ) Os algoritmos de busy-wait se baseiam no teste contínuo de uma condição.

d)( ) Condições de disputa ocorrem devido às diferenças de velocidade na execução dos processos.

e)( ) Condições de disputa ocorrem quando dois processos tentam executar o mesmo código ao mesmo tempo.

3. Sobre as afirmações a seguir, relativas aos **mecanismos de comunicação**, indique quais são incorretas, justificando sua resposta:

a)( ) As filas de mensagens POSIX são um exemplo de canal de comunicação com capacidade nula.

b)( ) A memória compartilhada provê mecanismos de sincronização para facilitar a comunicação entre os processos.

c)( ) A troca de dados através de memória compartilhada é mais adequada para a comunicação em rede. As filas de mensagens POSIX são um bom exemplo de canal de eventos.

d)( ) Nas filas de mensagens POSIX, as mensagens transitam através de arquivos em disco criados especialmente para essa finalidade.

e)( ) Em UNIX, um pipe é um canal de comunicação unidirecional que liga a saída padrão de um processo à entrada padrão de outro.

4. Em que circunstâncias o uso de espera ocupada é inevitável?

5. Explique como processos que comunicam por troca de mensagens se comportam em relação à capacidade do canal de comunicação, considerando as semânticas de chamada síncrona e assíncrona.

6. Considere o modelo clássico Leitor/Escritor, representado pelo código abaixo, usando semáforos iniciados com valor 1. Em seguida, avalie as afirmativas de I a IV.

```
 1 leitor() {
 2    down(R);
 3       rcount++;
 4       if (rcount==1) down(W);
 5    up(R);
 6    ler_dados();
 7    down(R);
 8       rcount--;
 9       if (rcount==0) up(W);
10    up(R);
11 }
```

```
1 escritor() {
2    produzir_dados()
3    down(W);
4       escrever_dados();
5    up(W);
6 }
```

I - Essa solução pode levar a starvation de escritores, pois enquanto houver um leitor em processamento (linha 6 do leitor), o semáforo W não será desbloqueado (linha 9 do leitor), e o escritor não poderá prosseguir (linha 3 do escritor)

II - O semáforo R é utilizado como uma forma de garantir a exclusão mútua entre os processos que tentarem modificar a contagem de leitores, e também alterar o semáforo W.

III - O semáforo R é utilizado como uma forma de garantir que somente um escritor esteja manipulando os dados, devendo então ser realizado o bloqueio somente dos leitores.

IV - Como escritores são mais frequentes que leitores, esta solução precisaria ser reescrita para incluir uma contagem de processos, adicionando um novo semáforo antes das linhas 3 e 5 do código do escritor.

Estão corretas as afirmativas:

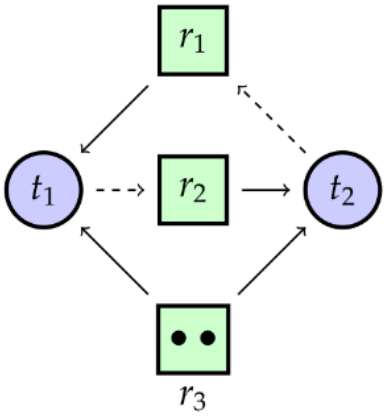
A. I e III
B. III e IV
C. I e II
D. I, II e IV
E. II e IV

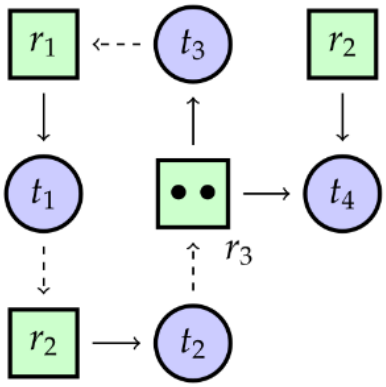
7. O trecho de código a seguir apresenta uma solução para o problema do jantar dos filósofos, mas ele está sujeito a impasses. Explique como o impasse pode ocorrer. A seguir, modifique o código para que ele funcione corretamente e explique sua solução.

```
#define N 5

sem_t garfo[5] ; // 5 semáforos iniciados em 1

void filosofo (int i) // 5 threads (i varia de 0 a 4)
{
    while (1)
    {
        medita ();
        sem_down (garfo [i]) ;
```

```
        sem_down (garfo [(i+1) % N]) ;
        come ();
        sem_up (garfo [i]) ;
        sem_up (garfo [(i+1) % N]) ;
    }
}
```

8. Nos grafos de alocação de recursos da figura a seguir, indique o(s) ciclo(s) onde existe um impasse:





9. O **Rendez-Vous** é um operador de sincronização forte entre dois processos ou threads, no qual um deles espera até que ambos cheguem ao ponto de encontro (*rendez-vous*, em francês). O exemplo a seguir ilustra seu uso:

```
Processo A
    A1 () ;
    rv_wait (rv) ;
    A2 () ;
    rv_wait (rv) ;
    A3 () ;
```

```
Processo B
    B1 () ;
    rv_wait (rv) ;
    B2 () ;
    rv_wait (rv) ;
    B3 () ;
```

Considerando a relação $a \rightarrow b$ como "a ocorre antes de b" e a relação $a \parallel b$ como "a e b ocorrem sem uma ordem definida", temos as seguintes restrições de sincronização:

- $\forall (i,j), A_i \rightarrow B_{j>i}$ e $B_i \rightarrow A_{j>i}$ (imposto pelo Rendez-Vous)
- $\forall (i,j), A_i \rightarrow A_{j>i}$ e $B_i \rightarrow B_{j>i}$ (imposto pela execução sequencial)
- $\forall (i,j), A_i \parallel B_{j=i}$ (possibilidade de execução concorrente)

Escreva o pseudo-código necessário para implementar Rendez-Vous, usando semáforos ou mutexes. Não esqueça de inicializar as variáveis e semáforos utilizados. Soluções que incorram em espera ocupada (busy wait) não devem ser usadas.

```
// estrutura que representa um RV
typedef struct rv_t
{
    ... // completar
} rv_t ;

// operador de espera no RV
void rv_wait (rv_t *rv)
{
```

```
    ... // completar
}

// inicialização do RV
void rv_init (rv_t *rv)
{
    ... // completar
}
```

10. Desenhe o diagrama de tempo da execução e indique as possíveis saídas para a execução concorrente das duas *threads* cujos pseudo-códigos são descritos a seguir. Os semáforos s1 e s2 estão inicializados com zero (0).

```
thread1 ()
{
    down (s1) ;
    printf ("A") ;
    up (s2) ;
    printf ("B") ;
}
```

```
thread2 ()
{
    printf ("X") ;
    up (s1) ;
    down (s2) ;
    printf ("Y") ;
}
```